MEANFLOW AND IMPROVED MEANFLOW FOR ONE-STEP GENERATIVE MODELING

Hjalte Bjørnstrup (fhz806)

Department of Computer Science, University of Copenhagen

ABSTRACT

Flow Matching is a generative modeling framework that learns how to move samples between a simple prior distribution, such as Gaussian noise, and the data distribution. The model learns an instantaneous velocity field, which normally has to be evaluated several times during generation. MeanFlow (MF) instead introduces an average velocity over a time interval. This makes it possible to move directly across a large part of the trajectory and enables one-step generation. This paper first describes the main idea and training objective of MeanFlow. It then discusses Improved MeanFlow (iMF), a direct follow-up that reformulates the training objective, introduces flexible classifier-free guidance, and improves how conditioning is handled. These changes improve the 1-NFE FID on ImageNet 256×256 from 3.43 for MeanFlow to 1.72 for iMF.

***Index Terms*—** Flow Matching, generative modeling, MeanFlow, one-step generation, Improved MeanFlow

1. INTRODUCTION

The goal of generative modeling is to learn how to generate new samples that follow the same distribution as the training data. Flow Matching (FM) does this by constructing paths between a simple prior distribution and the data distribution, and learning the velocity field along these paths [1].

During generation, the model starts from a sample of Gaussian noise and follows the learned velocity field towards the data distribution. The problem is that the velocity v is an instantaneous velocity: it only tells the model how to move at its current position and time. The model therefore normally has to evaluate the velocity field repeatedly while solving an ordinary differential equation (ODE). Each neural network evaluation is called a Number of Function Evaluation (NFE). A method using one network evaluation is therefore referred to as 1-NFE.

MeanFlow (MF) [1] approaches this problem by learning an average velocity u between two time steps instead of only learning the instantaneous velocity v. If the average velocity over the complete trajectory can be predicted correctly, generation can be performed using a single network evaluation.

The second paper considered in this work is Improved MeanFlow (iMF) [2]. It was chosen because it is a direct follow-up to MeanFlow and studies problems in the original training formulation. It keeps the main idea of learning an average velocity, but changes how the model is trained and how classifier-free guidance and other conditions are represented.

2. BACKGROUND: FLOW MATCHING

Flow Matching constructs a path between a data sample

$$x \sim p_{\text{data}}(x)$$

and a sample from a simple prior distribution

$$\epsilon \sim p_{\text{prior}}(\epsilon).$$

For image generation, the prior is typically Gaussian noise. A flow path can be written as

$$z_t = a_t x + b_t \epsilon, \tag{1}$$

where a_t and b_t define the schedule over time. A commonly used linear schedule is

$$a_t = 1 - t, \qquad b_t = t, \tag{2}$$

which gives

$$z_t = (1 - t)x + t\epsilon. \tag{3}$$

With this convention,

$$z_0 = x, \qquad z_1 = \epsilon.$$

The path therefore goes from data at $t = 0$ to noise at $t = 1$. Generation later runs in the opposite direction, from noise at $t = 1$ towards data at $t = 0$.

The instantaneous velocity of this path is the derivative of z_t with respect to time:

$$v_t = z_t' = a_t' x + b_t' \epsilon. \tag{4}$$

For the linear schedule in Eq. 3, this becomes

$$v_t = \epsilon - x. \tag{5}$$

This v_t is called the conditional velocity. It is easy to compute during training because both x and ϵ are known.

However, a particular point z_t can be produced by several different (x, ϵ) pairs. These pairs may have different conditional velocities. Flow Matching therefore considers the expected velocity over all possibilities:

$$v(z_t, t) \triangleq \mathbb{E}\left[v_t \mid z_t\right]. \tag{6}$$

This is called the marginal velocity. It is the velocity field that the neural network ultimately tries to represent.

A neural network v_θ can in principle be trained directly against the marginal velocity:

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}\left[||v_\theta(z_t, t) - v(z_t, t)||_2^2\right]. \tag{7}$$

Computing the marginal velocity explicitly is difficult because it requires averaging over all possible conditional paths. Instead, Flow Matching trains with the conditional velocity:

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}\left[||v_\theta(z_t, t) - v_t||_2^2\right]. \tag{8}$$

Minimizing this conditional objective is equivalent to minimizing the corresponding marginal Flow Matching objective [1].

After training, samples are generated by solving the ODE

$$\frac{d}{dt} z_t = v(z_t, t), \tag{9}$$

starting from

$$z_1 = \epsilon \sim p_{\mathrm{prior}}.$$

For two time steps $r < t$, the exact solution satisfies

$$z_r = z_t - \int_r^t v(z_\tau, \tau)\, d\tau. \tag{10}$$

In practice this integral normally has to be approximated using several discrete steps. Even though each conditional path can be chosen to be a straight line, the marginal velocity field can produce curved trajectories. This is why simply taking one large step using the instantaneous velocity can give a poor approximation.

3. MEANFLOW

3.1. Average velocity

The main idea of MeanFlow is to model the average velocity instead of only the instantaneous velocity. For two time steps $r < t$, MeanFlow defines

$$u(z_t, r, t) \triangleq \frac{1}{t - r} \int_r^t v(z_\tau, \tau)\, d\tau. \tag{11}$$

The difference between u and v is important. The instantaneous velocity $v(z_t, t)$ gives the direction and speed of the flow at one particular point in time. The average velocity $u(z_t, r, t)$ instead describes the average direction and speed of the flow over the interval $[r, t]$.

From Eq. 11, multiplying the average velocity by the interval length $(t - r)$ gives

$$(t - r)u(z_t, r, t) = \int_r^t v(z_\tau, \tau)\, d\tau. \tag{12}$$

Substituting this into Eq. 10 gives

$$z_r = z_t - (t - r)u(z_t, r, t). \tag{13}$$

This immediately gives the motivation for one-step generation. If the complete interval is used, with $r = 0$ and $t = 1$, then

$$z_0 = z_1 - u(z_1, 0, 1). \tag{14}$$

A model that accurately predicts $u(z_1, 0, 1)$ therefore does not need to follow the instantaneous velocity field through many smaller steps.

The challenge is that Eq. 11 contains an integral. Computing this integral during every training iteration would itself require repeatedly evaluating the velocity field. MeanFlow therefore needs a way to learn u without calculating this integral directly.

3.2. The MeanFlow identity

Starting from the definition of average velocity,

$$(t - r)u(z_t, r, t) = \int_r^t v(z_\tau, \tau)\, d\tau, \tag{15}$$

the authors differentiate both sides with respect to t, while treating r as independent of t. This gives

$$u(z_t, r, t) + (t - r)\frac{d}{dt} u(z_t, r, t) = v(z_t, t). \tag{16}$$

Rearranging gives the central MeanFlow identity:

$$u(z_t, r, t) = v(z_t, t) - (t - r)\frac{d}{dt} u(z_t, r, t). \tag{17}$$

This identity relates the quantity MeanFlow wants to learn, u, to the instantaneous velocity v that is already available from Flow Matching.

The derivative in Eq. 17 is a total derivative. Following the paper, it can be expanded as

$$\frac{d}{dt} u(z_t, r, t) = v(z_t, t)\partial_z u + \partial_t u. \tag{18}$$

This can be computed efficiently as a Jacobian-vector product (JVP). The JVP uses the Jacobian of u with respect to (z, r, t) together with the tangent vector

$$[v, 0, 1].$$

3.3. Training MeanFlow

MeanFlow uses a neural network

$$u_\theta(z_t, r, t)$$

to approximate the average velocity. The MeanFlow identity gives the following training target:

$$u_{\text{tgt}} = v(z_t, t) - (t - r)\left(v(z_t, t)\partial_z u_\theta + \partial_t u_\theta\right). \tag{19}$$

As in standard Flow Matching, the marginal velocity $v(z_t, t)$ is not directly computed. It is replaced by the conditional velocity v_t. The practical target is therefore

$$u_{\text{tgt}} = v_t - (t - r)\left(v_t\partial_z u_\theta + \partial_t u_\theta\right). \tag{20}$$

For the default linear flow,

$$v_t = \epsilon - x.$$

The network is then trained with

$$\mathcal{L}(\theta) = \mathbb{E}\left[||u_\theta(z_t, r, t) - \text{sg}\left(u_{\text{tgt}}\right)||_2^2\right]. \tag{21}$$

Here, sg denotes stop-gradient. This means that u_{tgt} is treated as a fixed target when the network parameters are updated.

Conceptually, MeanFlow remains close to ordinary Flow Matching. A data sample x, noise ϵ, and time values (r, t) are sampled. The intermediate state z_t and conditional velocity v_t are known. The main difference is that MeanFlow modifies the target using the derivative term in Eq. 20 so that the network learns an average velocity rather than only an instantaneous one.

For ImageNet 256×256, the largest reported MeanFlow model achieves an FID of 3.43 using only 1-NFE generation [1]. The model is trained from scratch and does not require distillation or a pretrained multi-step teacher.

4. IMPROVED MEANFLOW

Improved MeanFlow [2] directly builds on the MeanFlow formulation above. The authors keep the average velocity u and the same one-step sampling idea, but identify two main problems with the original method. First, the target u_{tgt} depends partly on the network u_θ itself. Second, classifier-free guidance uses a guidance scale that must be fixed during training.

The Improved MeanFlow paper writes the noise variable as e instead of ϵ and often leaves the (r, t) arguments out of u and v for brevity. This section follows that notation where useful.

4.1. Reformulating MeanFlow as a v-loss

The original MeanFlow identity can equivalently be written as

$$v(z_t) = u(z_t) + (t - r)\frac{d}{dt}u(z_t). \tag{22}$$

This changes how the training problem can be viewed. Instead of treating u as the regression target, iMF uses the instantaneous velocity v as the target, as in standard Flow Matching.

The right-hand side of Eq. 22 is represented by the compound prediction

$$V_\theta \triangleq u_\theta(z_t) + (t - r)\,\text{JVP}_{\text{sg}}\left(u_\theta; e - x\right). \tag{23}$$

This gives the equivalent v-loss

$$\mathbb{E}\left[||V_\theta - (e - x)||_2^2\right]. \tag{24}$$

Writing MeanFlow in this form reveals a problem. The prediction V_θ depends not only on the noisy state z_t, but also on the conditional velocity $e - x$. The paper writes this explicitly as

$$V_\theta(z_t, e - x).$$

This is unusual for a regression problem because $e - x$ is the quantity used as the training target in Eq. 24. More importantly, the JVP should theoretically use the marginal velocity $v(z_t)$ as its tangent direction, not the conditional velocity $e - x$.

4.2. Improved parameterization

iMF therefore replaces the conditional velocity inside the JVP with a network prediction $v_\theta(z_t)$:

$$V_\theta(z_t) \triangleq u_\theta(z_t) + (t - r)\,\text{JVP}_{\text{sg}}\left(u_\theta; v_\theta\right). \tag{25}$$

Now the prediction depends only on z_t, which gives a more standard regression setup.

A separate network does not necessarily have to be trained for v_θ. From the definition of average velocity,

$$v(z_t, t) \equiv u(z_t, t, t). \tag{26}$$

When the interval shrinks to zero, average and instantaneous velocity are the same. iMF can therefore reuse the MeanFlow network and define

$$v_\theta(z_t, t) = u_\theta(z_t, t, t). \tag{27}$$

The authors also investigate an auxiliary v-head, which gives the model additional capacity for predicting v during training. This head is not needed during inference.

The practical difference between MF and iMF is therefore small in notation but important during training. Original MF

feeds the noisy conditional velocity $e - x$ into the JVP. iMF instead uses the model's estimate $v_\theta(z_t)$. The paper finds that the original formulation has a higher-variance and less stable training loss, while the new formulation produces more stable optimization [2].

4.3. Flexible classifier-free guidance

The second main change concerns classifier-free guidance (CFG). In original MeanFlow, the guided velocity field is

$$v_{\text{cfg}}(z_t \mid \mathbf{c}) = \omega v(z_t \mid \mathbf{c}) + (1-\omega)v(z_t), \tag{28}$$

where $\mathbf{c}$ is the class condition and ω is the guidance scale. The original method fixes ω during training. This allows guidance to be included while preserving 1-NFE sampling, but it means that the guidance strength cannot freely be changed after training.

iMF instead treats ω as another condition of the network:

$$V_\theta(\cdot \mid \mathbf{c}, \omega) \triangleq u_\theta(z_t \mid \mathbf{c}, \omega) + (t-r)\,\text{JVP}_{\text{sg}}\,. \tag{29}$$

During training, different values of ω are sampled. The guidance scale can then be changed at inference time without increasing the number of network evaluations.

The paper further groups guidance-related conditions as

$$\Omega = \{\omega, t_{\min}, t_{\max}\}\,. \tag{30}$$

The complete MeanFlow network can then be written as

$$u_\theta = u_\theta\left(z_t \mid r, t, \mathbf{c}, \Omega\right). \tag{31}$$

4.4. In-context conditioning and results

iMF also changes how these conditions are given to the network. Instead of combining all conditions using adaLN-zero, each condition is represented by several tokens. These tokens are concatenated with the image latent tokens and processed together by the Transformer.

This makes it easier to handle the different conditions and also reduces the model size. For example, the Base model is reduced from 133M to 89M parameters when adaLN-zero is replaced with the proposed in-context conditioning [2].

The final iMF-XL/2 model achieves an FID of 1.72 on ImageNet 256×256 with 1-NFE generation. In comparison, the original MeanFlow-XL/2 reports 3.43 FID with the same 1-NFE setting. iMF-XL/2 also uses 610M parameters compared with 676M for the original MeanFlow-XL/2 [1, 2].

5. DISCUSSION

The main difference between Flow Matching and MeanFlow is the quantity learned by the network. Flow Matching learns the instantaneous velocity v. This target is simple to obtain during training, but several velocity evaluations may be needed to follow a curved trajectory during inference.

MeanFlow instead learns the average velocity u over an interval. The MeanFlow identity makes it possible to train this quantity without explicitly evaluating the integral that defines it. Once u has been learned, the whole interval from noise to data can be approximated in one step.

Improved MeanFlow keeps this main idea. Its contribution is mainly to make the training formulation cleaner and more stable. By rewriting MeanFlow as a v-loss and using the predicted marginal velocity v_θ inside the JVP, the network prediction no longer depends directly on the conditional target $e - x$. iMF also makes classifier-free guidance adjustable at inference time and introduces a lighter in-context conditioning architecture.

The development from FM to MF and finally iMF can therefore be seen as a progression from learning local movement, to learning movement over a whole interval, and finally to improving how this interval-based model is trained. The empirical results show that these changes can preserve 1-NFE generation while substantially improving generation quality.

6. REFERENCES

[1] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J. Zico Kolter, and Kaiming He, "Mean Flows for One-step Generative Modeling," May 2025, arXiv:2505.13447 [cs.LG].

[2] Zhengyang Geng, Yiyang Lu, Zongze Wu, Eli Shechtman, J. Zico Kolter, and Kaiming He, "Improved Mean Flows: On the Challenges of Fastforward Generative Models," May 2026, arXiv:2512.02012 [cs.CV].